

Trabalho Final de ATR (2026/1)

1. INTRODUÇÃO E CONTEXTO

A inspeção regular de túneis e estruturas civis é essencial para garantir a segurança pública, mas métodos manuais são custosos e perigosos. O uso de robôs autônomos equipados com inteligência incorporada (*embodied intelligence*) permite que esses sistemas interajam com o ambiente em tempo real para otimizar a detecção de falhas.

Este trabalho consiste em desenvolver um sistema de inspeção de integridade estrutural. O objetivo é mapear o perfil do teto de um túnel para detectar anomalias superficiais, como buracos ou saliências, que podem indicar riscos estruturais.



(a) Illustration of the tunnel inspection



(b) Testing in the tunnel

Figura 1 - Ilustração de uma inspeção automática de túneis. Fonte: Altinay K, Ye Z, Mitoulis S-A, Ninić J. Automated surface damage detection with an embodied intelligence robotic platform. Data-Centric Engineering. 2026;7:e1. doi:10.1017/dce.2025.10033

Para simplificar a implementação, o problema será tratado em um plano 2D, com vista lateral.

2. OBJETIVOS

Aplicar os conhecimentos da disciplina Automação em Tempo Real na implementação de um sistema de inspeção automática de túneis, conforme a arquitetura proposta, contemplando tanto o simulador dos sensores quando o sistema de controle embarcado do robô de inspeção.

Objetivos específicos:

- Implementar um simulador que emule a física e os sensores do robô de inspeção.
- Implementar um sistema multitarefa para navegação e inspeção autônoma.
- Utilizar mecanismos de sincronização (mutexes, semáforos, variáveis de condição) para gerenciar o acesso a dados compartilhados e disparar eventos.
- Integrar a comunicação entre o robô e um centro de operação via protocolo MQTT.

3. ORGANIZAÇÃO DO DOCUMENTO

O Capítulo 4 apresenta as especificações gerais do sistema, que deve ser desenvolvido em duas etapas. O Capítulo 5 detalha as entregas da Etapa 1 e da Etapa 2. O Capítulo 6 apresenta instruções gerais para o desenvolvimento.

4. ESPECIFICAÇÕES GERAIS DO SISTEMA

O seu grupo foi contratado para desenvolver a aplicação embarcada no sistema de inspeção automática de túneis. A Figura 2 ilustra o problema de forma geral, em que o robô deve ser programado tanto para fazer a navegação quanto para a inspeção do túnel.

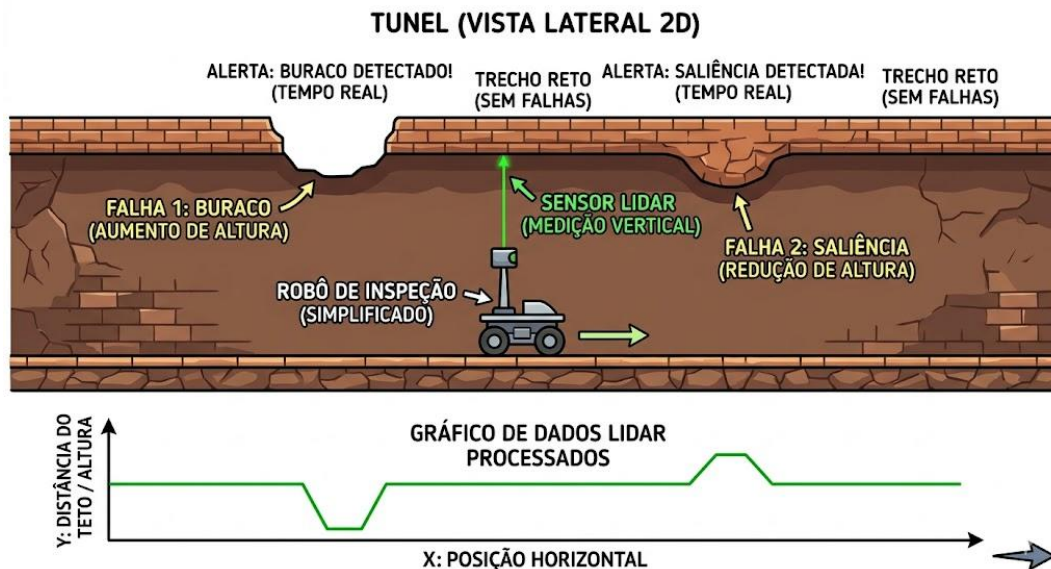


Figura 2 - Ilustração do ambiente simplificado de inspeção automática do robô.

O sistema completo é ilustrado na Figura 1, em que cada tarefa é identificada com um retângulo, e a interação entre elas é representada por setas.

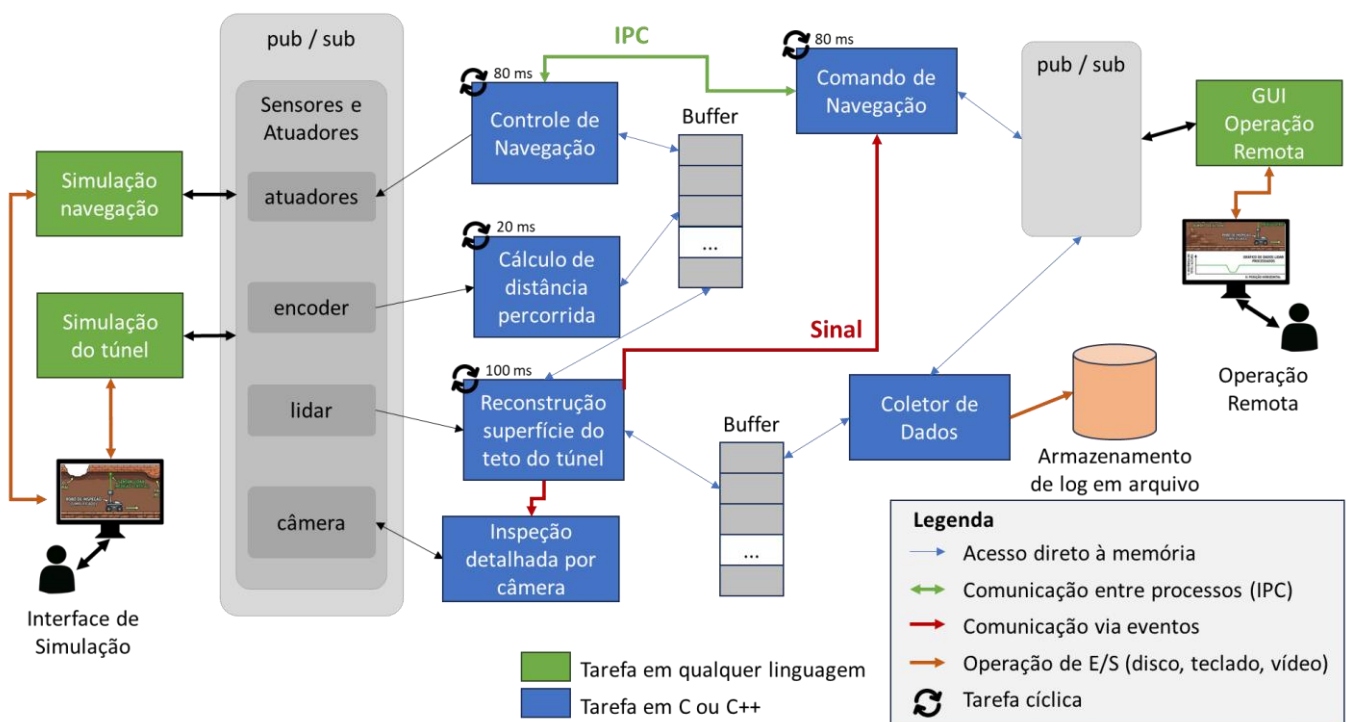


Figura 3 - Sistema simplificado de controle de um robô autônomo de inspeção de túneis.

O sistema será composto pelas seguintes tarefas executando em paralelo:

1. **Comando de Navegação:** tarefa que recebe comandos do sistema de operação remoto e traduz os comandos em setpoint de velocidade e liga/ desliga para o controle de navegação. O comando de navegação que deve implementar a lógica de manual/ automático.
2. **Controle de Navegação:** implementação de um controlador PID responsável pelo acionamento dos motores (controle de velocidade) a partir de um *setpoint* de velocidade recebido pelo Comando de Navegação.
3. **Cálculo de distância percorrida:** tarefa que lê os dados do encoder para calcular a distância percorrida. O encoder deve ter funcionamento simulado em que gera uma troca de estado (binário, $0 \rightarrow 1$ ou $1 \rightarrow 0$) a cada metro percorrido.
4. **Reconstrução da superfície do teto do túnel:** tarefa que lê os dados de distância e posição, aplicando filtros de média móvel para reduzir ruídos, para colocar no buffer que vai para o coletor de dados. Ao perceber uma variação severa (parâmetro que deve ser configurável pela operação remota) da superfície do teto, essa tarefa deve enviar uma sinalização para que o robô ande mais devagar e ligue a câmera para realizar a inspeção visual da falha.
5. **Inspeção detalhada por câmera:** tarefa que fica aguardando ser acionada e, quando estiver ativa, roda um processamento pesado, simulando o acionamento de uma câmera e análise da imagem por algoritmo de inteligência incorporada. Para simular essa parte você deve rodar algo que efetivamente use o processador e não apenas um “sleep”.
6. **Coletor de dados:** registra a posição e a superfície do teto. Todos os registros devem ter *timestamp*, localização (x, y) e nível de confiança. Quanto mais medições próximas, maior o nível de confiança. Essa análise deve ser feita online no coletor. As informações devem ser armazenadas em disco e também disponibilizadas para o sistema de operação remota.
7. **Operação remota:** código implementado em qualquer linguagem que mostra o comportamento do robô, com todos os dados disponíveis. Deve ser disponibilizado também na tela todos os comandos possíveis para o robô.
8. **Simulação (de navegação e do túnel):** código implementado em qualquer linguagem que simula o ambiente físico. O código deve ler o valor dos atuadores e gerar o comportamento dos sensores, simulando um ambiente de navegação (implemente as leis da mecânica clássica de Newton) com ruído de medição. Esse código pode ser implementado em uma ou mais tarefas. A simulação deve ter uma interface gráfica para permitir visualizar o todo.

Esse projeto será desenvolvido em duas etapas complementares:

- **Etapa 1:** definição da arquitetura completa do sistema e implementação de todas as tarefas em azul na Figura 1. Definição e implementação dos buffers e suas interações com as demais tarefas. As interações definidas pelas setas em vermelho (eventos) laranja (I/O) e em verde (IPC) não são implementadas nessa etapa. O servidor *publisher* e *subscriber* e as interfaces gráficas de Operação Remota e Simulação também não são implementadas nessa etapa.
- **Etapa 2:** implementação das interfaces de **Operação Remota** e **Simulação**. Essa implementação não precisa ser necessariamente em C++, você pode usar uma linguagem que simplifique o processo (e.g. Python com pygame). Implementação dos servidores e clientes MQTT (*publisher* e *subscriber*) para comunicação com os sistemas. Relatório com análise de tempo de resposta e escalonabilidade das tarefas do sistema, incluindo testes.

A seguir serão apresentados com mais detalhes cada módulo do sistema. Esta é uma especificação básica de funcionamento do sistema. Existem várias nuances e detalhes que serão definidos por você e seu grupo para garantir a funcionalidade do sistema.

A Tabela 1 mostra os sensores e atuadores disponíveis no caminhão.

Tabela 1 - Sensores e atuadores do robô autônomo.

	Nome da variável	Tipo	Descrição
I	i_encoder	bool	Variável que simula a entrada de um encoder, que troca de estado a cada metro percorrido pelo robô.
I	i_lidar	int	Resposta do sensor LIDAR do veículo exibindo a distância no eixo y, com relação à altura do robô.
O	o_liga_camera	bool	Comando para ligar a câmera e realizar fotos da falha detectada na superfície
O	o_aceleracao	int	Determina a aceleração do veículo em percentual (-100 a 100%)

Os comandos do robô são definidos pelo sistema de Operação Remota. A Tabela 2 mostra os possíveis estados (para exibição na interface gráfica) e comandos (para acionamento remoto pelo operador).

Tabela 2 - Estados e comandos para o robô. Os estados são gerados pelo Comando de Navegação e os comandos são gerados pela interface.

	Nome da variável	Tipo	Descrição
E	e_inspecao	bool	Estado que indica falha detectada na superfície e o robô está tirando fotos e navegando com velocidade limitada (1: falha, 0: sem falha)
E	e_automatico	bool	Estado que identifica o modo de operação do robô (0: manual, 1: automático)
C	c_automatico	bool	Comando para passar o robô para o modo automático (true). O reset desse comando
C	c_man	bool	Comando para passar o robô para o modo manual (true).
J	j_sp_velocidade	int	Setpoint de velocidade do robô para o controlador de velocidade.
C	c_direita	-	Comando para acelerar o robô para a direita.

C	c_esquerda	-	Comando para acelerar o robô para a esquerda.
C	c_para	-	Comando para parar o robô.

Atenção: essa organização de variáveis é sugestiva, se quiser pode organizar essas informações de forma diferente no seu código, por exemplo, juntando vários valores booleanos numa variável inteira.

5. ENTREGAS DO TRABALHO

As entregas estão especificadas a seguir.

- 1) **ETAPA 1 (18/05/2026)**: definição da arquitetura
 - a) Figura da arquitetura detalhada do sistema, especificando linguagens de programação, ferramentas de sincronização, APIs e demais detalhes que serão implementados.
 - b) Relatório em PDF com uma apresentação parcial do sistema. Descreva em detalhe a implementação dos buffers e mostre os testes de troca de dados implementadas. Foque a sua explicação nos mecanismos de sincronização utilizados.
- 2) **ETAPA 2 (15/06/2026)**: sistema completo
 - a) Todos os arquivos do sistema.
 - b) Relatório em PDF com descrição geral do sistema e explicando partes críticas do código, com foco nas ferramentas lecionadas na disciplina. O relatório deve conter uma análise de tempo de resposta e escalonabilidade, respondendo a seguinte questão: para um dado sistema computacional, qual o menor tempo que podemos utilizar nas tarefas cíclicas de modo a garantir a escalonabilidade do sistema?
 - c) Vídeo de apresentação do sistema funcionando (até o máximo de 10 min).
 - d) Apresentação do código diretamente para o professor.

6. INSTRUÇÕES GERAIS

1. O trabalho pode ser feito por grupos de **até 3 pessoas**. Na apresentação do código, será perguntado o papel de cada integrante no trabalho.
2. O trabalho deve ser submetido via Moodle, dentro do prazo de entrega, como um **arquivo ZIP** contendo todo o conteúdo.
3. Cuidado com referências absolutas a pastas, use sempre referências relativas de onde o programa principal está rodando.
4. Cuide da documentação. A sua documentação, em **formato PDF**, deve ter **instruções de como compilar** e como rodar o código.
5. Faça com que todo o código execute a partir de um único arquivo, executando todos os processos necessários.
6. Use ferramentas e conceitos de programação e desenvolvimento de sistemas para fazer uma aplicação com o máximo de profissionalismo possível, com boa organização.
7. Teste cada módulo do seu programa e cada interação com outros módulos. Faça vários testes cuidadosos no código.
8. Ponto extra: o seu trabalho **pode receber até 5 pontos extras**, a critério do professor, caso implemente melhorias, acrescentando outros módulos no programa (ex. implementar sensor IMU simulado para ser aplicado em túneis com declive, implementação da simulação e do sistema de inspeção visual usando YOLO para detecção de eventuais objetos no teto).

Item	Pontuação
Entrega da Etapa 1 e da Etapa 2 no prazo	25
Entrega apenas da Etapa 2	15
Entrega apenas da Etapa 1	0
+Extra: túnel com declive e sensor IMU	até +2 pts
+Extra: inspeção visual com YOLO	até +3 pts